

Lung Cancer Classification Using CT-Derived Radiomic and Clinical Features

July 23, 2026

Abstract

This paper investigates lung cancer classification from CT-derived radiomic and clinical features using a classical machine learning workflow. The dataset is derived from the LIDC-IDRI collection and organized into prepared feature tables with different standard-deviation settings. The study combines data verification, preprocessing, feature selection, and model evaluation to predict malignancy-related labels from structured tabular inputs.

The main machine learning approach uses feature selection methods such as recursive feature elimination, tree-based importance, variance thresholding, and Lasso, followed by classification with Random Forest and K-Nearest Neighbors models. The experiments show that model performance depends strongly on the selected feature set, with KNN sweeps identifying different best values of k across runs and feature-selection strategies achieving cross-validation scores in the mid-to-high 0.6 range. A representative KNN evaluation produced class-wise precision of 0.57, 0.79, 0.50, and 0.00, and an overall confusion matrix that shows strong performance on the major classes but weak prediction for one class. The results indicate that classical machine learning is a viable baseline for the task, but class imbalance, missing data, and limited representation of some classes remain important constraints.

1 Introduction

Lung cancer remains one of the leading causes of cancer mortality worldwide, and early diagnosis is critical for improving patient outcomes. CT imaging plays a central role in detection and follow-up because it can reveal nodule characteristics that are not easily captured by routine clinical examination. However, interpretation of CT scans is challenging, especially when nodules are subtle, heterogeneous, or ambiguous across radiologists.

Machine learning provides a practical way to support medical imaging analysis by learning predictive patterns from radiomic descriptors and structured clinical annotations. In this project, the goal is not to replace clinical judgment, but to investigate whether a classical machine learning pipeline can learn useful patterns from the LIDC-IDRI-derived feature tables and provide a reproducible benchmark for later work.

The objectives of the project are:

1. Build a clean and reproducible lung cancer classification workflow.
2. Compare multiple feature selection strategies on the prepared CT feature tables.
3. Evaluate classical machine learning models using standard classification metrics.
4. Document the results in a form that can be extended to deeper models or image-based approaches later.

The scope of the work is limited to structured tabular features extracted from CT-related metadata and radiomic summaries. Raw image segmentation, deep convolutional learning, and clinical deployment are outside the present scope.

The paper is organized as follows: Section 2 defines the classification problem. Section 3 reviews related work. Section 4 describes the dataset and its preparation. Section 5 explains the methodology. Section 6 details the experimental setup. Section 7 defines the evaluation metrics. Section 8 presents the results. Section 9 discusses the findings. Section 10 covers limitations. Section 11 addresses ethical considerations. Section 12 proposes future work. Section 13 concludes the paper.

2 Problem Statement

This project addresses a multi-class classification problem where the objective is to predict malignancy-related labels from CT-derived tabular features. The input consists of structured radiomic measurements and nodule-level diagnostic fields. The target variable is the malignancy label used throughout the notebooks.

The expected output is a predicted class for each sample, together with class probabilities or confidence scores where supported by the model. The problem is difficult for several reasons: the data contains missing values, some feature groups are redundant, the classes are not uniformly represented, and nodule-level labels may be ambiguous or incomplete. In addition, different feature-selection strategies produce different feature subsets, so model behavior can vary depending on how the input space is reduced.

3 Related Work

Lung cancer prediction with machine learning has been studied using both image-based and tabular radiomics workflows. Radiomics-based approaches are particularly attractive because they transform medical images into numerical descriptors that can be used with classical classifiers. This often reduces computational cost compared with end-to-end deep learning and improves interpretability for small to medium-sized datasets.

Deep learning methods, especially CNNs, have shown strong performance when trained on large image corpora, but they typically require more data, more compute, and more extensive validation. Classical machine learning remains important in medical research because it is easier to inspect, faster to train, and often more suitable when the available data is limited or already represented as features.

The LIDC-IDRI dataset is a common benchmark in this area because it combines CT scans, nodule annotations, and malignancy-related labels. Prior work on this dataset has used both handcrafted radiomic features and deep networks. The approach chosen here is motivated by the project structure and the available prepared feature tables: the data is already tabular, so a classical machine learning pipeline is a natural and reproducible starting point.

4 Dataset

4.1 Dataset Description

The project uses data derived from the LIDC-IDRI collection hosted by the Cancer Imaging Archive (TCIA). LIDC-IDRI contains thoracic CT scans with lesion annotations contributed by multiple radiologists. The repository includes prepared feature tables named `data_0.5.csv`, `data_1.0.csv`, and `data_1.5.csv`, along with source metadata under `data/`.

The target variable used in the notebooks is the malignancy label. The feature tables contain radiomic and diagnostic descriptors extracted from CT nodules and related metadata fields.

The exact number of samples and features depends on the table and preprocessing stage. Based on the notebook outputs, the data is large enough to support multiple feature-selection experiments, but not so large that classical models become impractical.

4.2 Data Preparation

Initial inspection focused on identifier consistency, duplicates, whitespace issues, and missing values. The verification notebook checks ID formatting, duplicate IDs, cross-table consistency, and missing data patterns.

Missing values are concentrated in several diagnostic fields, including nodule-level diagnosis and diagnosis method columns. The notebooks also show that some feature columns are entirely zero-valued, which indicates either absent information or encoded unknowns. This makes careful preprocessing necessary before modeling.

The class distribution is not perfectly balanced, and this is visible in the classification results, where one class is not predicted at all in a representative run. That imbalance affects precision, recall, and confusion-matrix behavior.

5 Methodology

5.1 Overall Pipeline

The workflow is summarized in Figure 1:

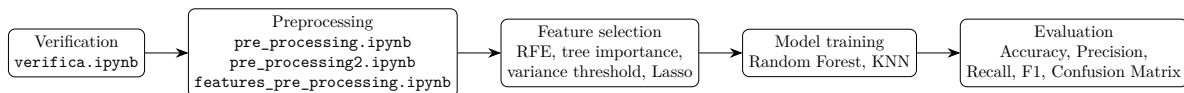


Figure 1: Notebook-based workflow used in the project.

The workflow has five main stages: data verification, preprocessing, feature engineering, feature selection, and model training/evaluation.

5.2 Data Preprocessing

Preprocessing includes loading the prepared CSV files, harmonizing column names, inspecting missing values, and cleaning diagnostic fields. Because the notebooks are organized into separate stages, the preprocessing notebooks are used as dependencies for later feature-selection and modeling notebooks.

The current workflow performs data validation and cleaning rather than heavy outlier engineering. Missing values are inspected and reported, and the feature tables are normalized into a reusable tabular format. Some columns are filtered or interpreted as unknown/zero-valued entries.

5.3 Feature Engineering

Feature engineering in this project is mostly driven by the existing radiomic and clinical descriptors in the prepared tables. The notebooks perform label handling and column standardization rather than generating large numbers of new synthetic variables.

Feature encoding is handled through preprocessing of diagnosis labels and consistent column naming. Feature scaling is not emphasized in the current notebook workflow because the primary models include tree-based methods, although scaled features would be useful for future distance-based or neural approaches.

5.4 Feature Selection

Four main selection strategies are used:

1. Recursive Feature Elimination with cross-validation (RFECV) using Random Forest.

2. Tree-based feature importance with thresholds of 0.01 and 0.02.
3. Variance threshold filtering.
4. Lasso-based sparse selection.

These methods were selected because they provide complementary views of feature relevance: recursive elimination searches for a performant subset, tree importance identifies strong nonlinear predictors, variance threshold removes low-information features, and Lasso keeps only features with non-zero linear contribution.

The notebooks show that the best cross-validation scores occur in the mid-to-high 0.6 range depending on the feature set. Tree-based and Lasso-based subsets were among the strongest performers in the comparison plot.

5.5 Machine Learning Models

5.5.1 Random Forest

Random Forest is used as the main classifier for feature selection and evaluation because it handles nonlinear relationships, is robust to mixed feature types, and provides feature importance estimates.

- **Advantages:** Works well on tabular data; supports feature importance analysis; handles nonlinear interactions.
- **Limitations:** Can be less interpretable than simpler linear models; sensitive to class imbalance if not tuned; may overfit if feature selection is weak.

5.5.2 K-Nearest Neighbors

KNN is used as a comparison model and as a small hyperparameter sweep over k . It provides a simple baseline and helps illustrate how local similarity behaves on the prepared feature tables.

- **Advantages:** Simple and intuitive; useful baseline for tabular similarity structure.
- **Limitations:** Sensitive to scaling and irrelevant features; can degrade quickly in high dimensions; computational cost grows with the training set.

6 Experimental Setup

6.1 Development Environment

The project is implemented in Python and organized in Jupyter notebooks. The main libraries used are pandas, NumPy, matplotlib, seaborn, and scikit-learn. The repository currently documents Python package dependencies in `requirements.txt`.

Hardware details are not recorded in the notebook outputs, so the exact machine specification is not reported here.

6.2 Training Procedure

- The notebooks use a train/test split with `test_size=0.2` and `random_state=42`.
- Cross-validation uses stratified folds to preserve class proportions during evaluation.
- Random Forest models are commonly initialized with `random_state=42` for reproducibility, and KNN is evaluated over several values of k .
- Random Forest remains the core estimator for feature selection and comparison.

6.3 Evaluation Strategy

The evaluation protocol compares multiple feature sets under the same classification setting. Performance is measured with cross-validation and held-out test results where available. This makes it possible to compare feature-selection methods on a common basis.

Reproducibility is supported by fixed random seeds, notebook-based execution, and explicit dependency listing. The notebooks are organized so that the preprocessing dependencies can be loaded from later stages.

7 Evaluation Metrics

The project uses the following metrics:

- Accuracy: overall proportion of correct predictions.
- Precision: how many predicted positives are correct.
- Recall: how many true positives are recovered.
- F1-score: harmonic mean of precision and recall.
- ROC-AUC: useful for ranking quality when probability estimates are available.
- Confusion matrix: shows class-by-class mistakes and reveals failure modes.

These metrics were chosen because medical classification problems require more than a single accuracy number. In particular, precision and recall are important when a minority class is clinically meaningful, and the confusion matrix helps identify whether the model is collapsing into majority-class predictions.

8 Results

8.1 Model Performance

The notebook experiments show that performance varies with the selected feature set. KNN sweeps identified different best values of k across runs, including $k = 4$ and $k = 6$ in separate comparisons. This indicates that the local neighborhood structure of the data changes materially with the underlying feature subset.

Feature-selection experiments produced the strongest cross-validation scores in the mid-to-high 0.6 range. Tree-based and Lasso-based subsets were especially competitive, while RFE remained stable across the three prepared data tables.

8.2 Feature Importance

The most relevant features depend on the selection strategy. RFE returns compact subsets tailored to the classifier objective, tree-based importance highlights features with high nonlinear predictive value, and Lasso keeps sparse linear predictors. The comparison suggests that no single feature subset dominates all criteria, but tree-based and sparse selections give the best overall balance in the notebook runs.

8.3 Visual Results

The repository includes exported figures from the notebook outputs, including KNN performance sweeps, confusion matrices, and feature-selection comparison plots. Figure 2 shows the KNN sweep, Figure 3 shows the confusion matrix, Figure 4 shows the annotated confusion matrix, Figure 5 shows the pairplot, and Figure 6 shows the feature-selection comparison.

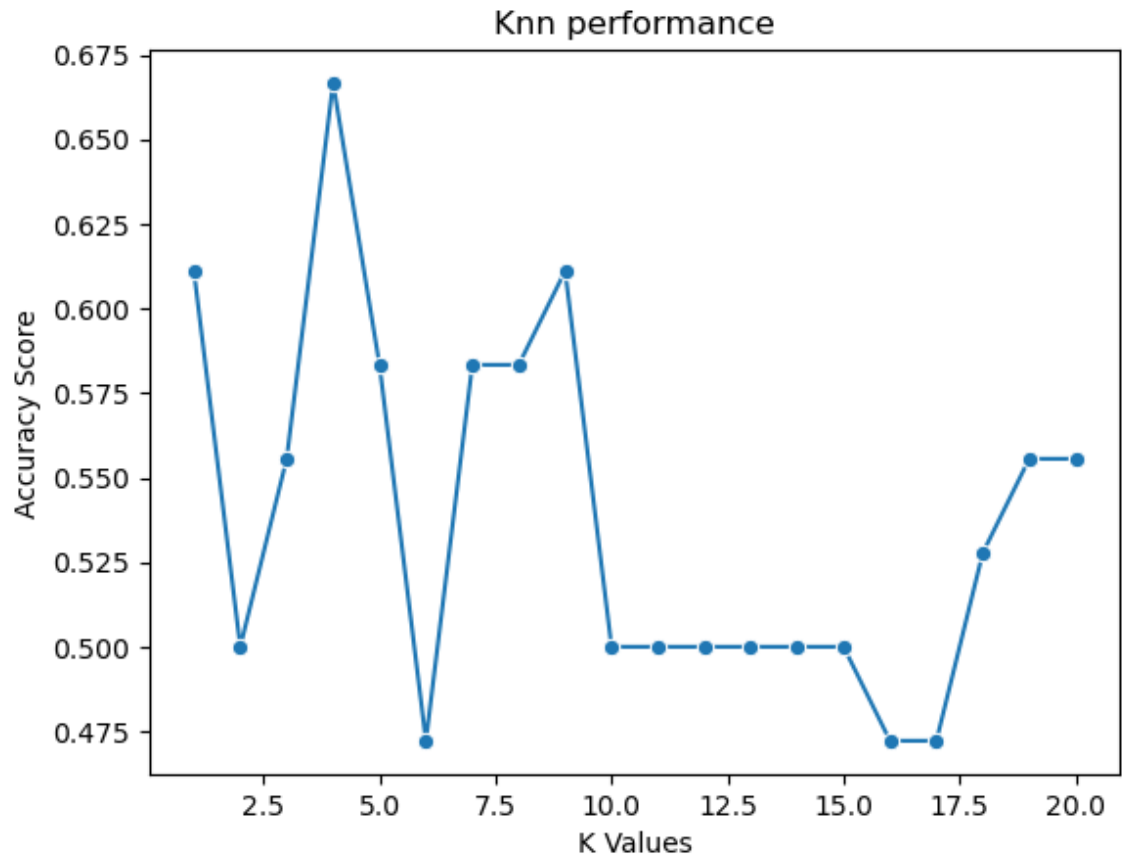


Figure 2: KNN performance sweep.

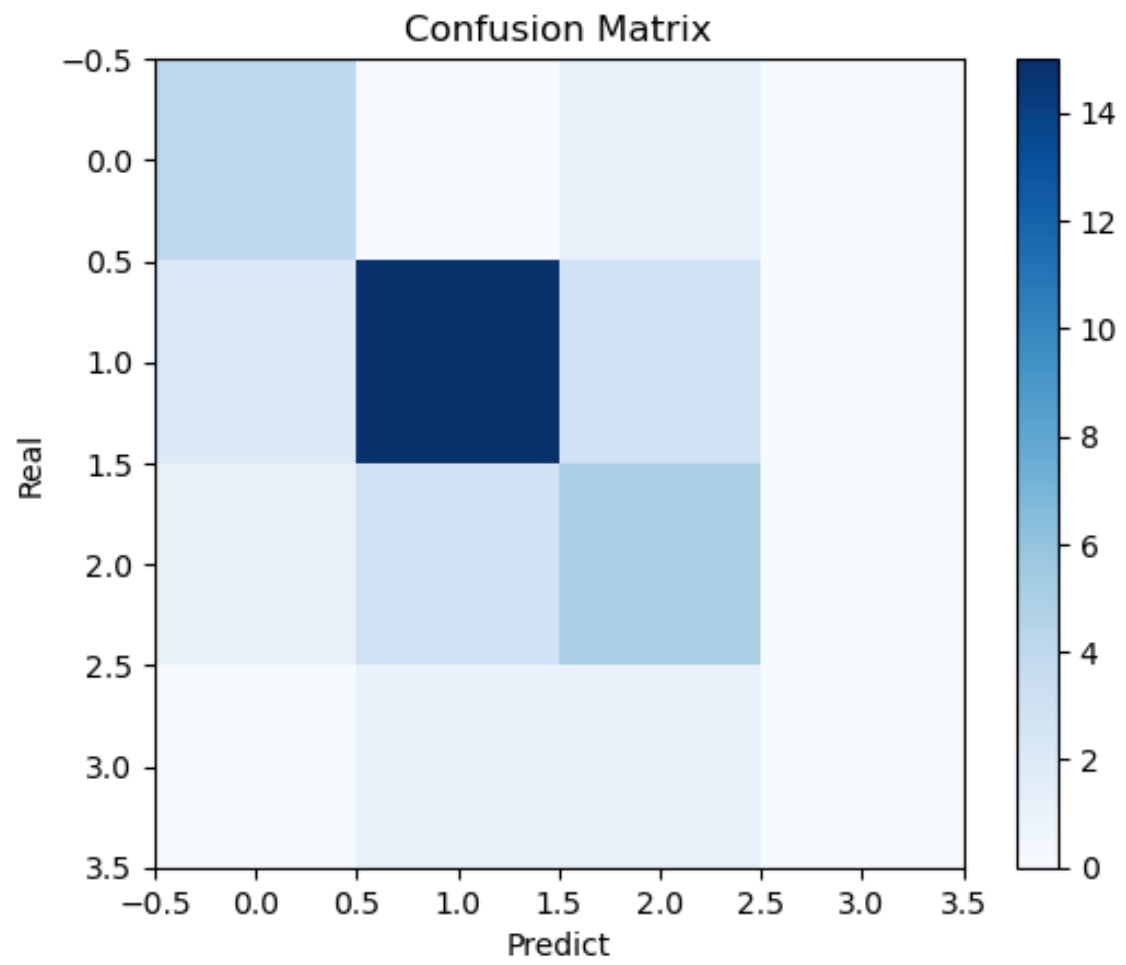


Figure 3: Confusion matrix.

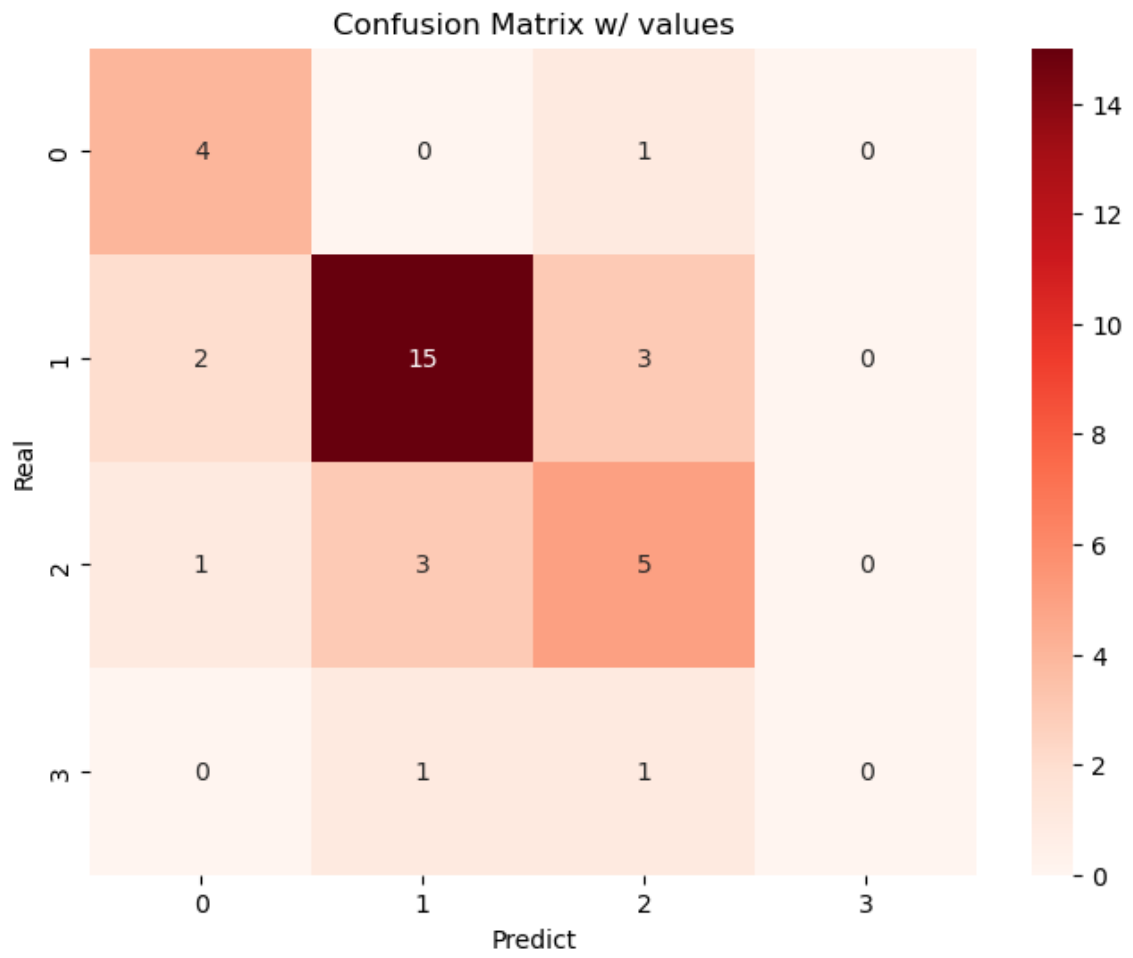


Figure 4: Confusion matrix with values.

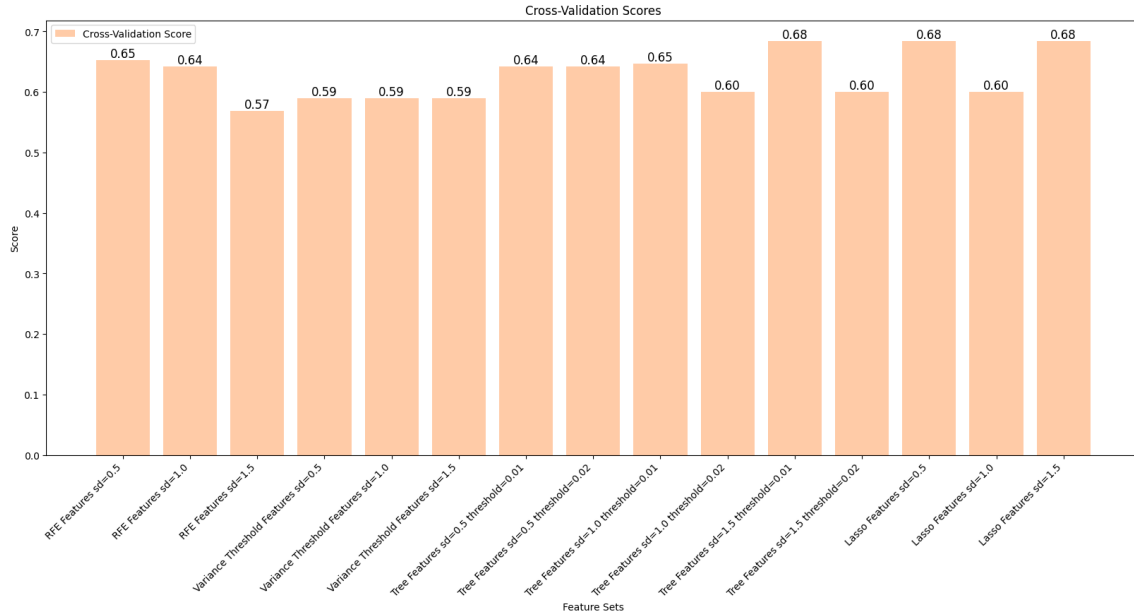


Figure 6: Feature selection comparison.

Table 1 summarizes a representative KNN evaluation.

	Pred 0	Pred 1	Pred 2	Pred 3
Real 0	4	0	1	0
Real 1	2	15	3	0
Real 2	1	3	5	0
Real 3	0	1	1	0

Table 1: Representative confusion matrix from the notebook evaluation.

Table 2 shows the same run’s class-wise scores.

Metric	Class 0	Class 1	Class 2	Class 3
Precision	0.57	0.79	0.50	0.00
Recall	0.80	0.75	0.56	0.00
F1-score	0.67	0.77	0.53	0.00

Table 2: Representative class-wise scores from the notebook evaluation.

8.4 Model Comparison

Random Forest appears to be the most useful model family in the current project because it supports both feature selection and classification, and it performs consistently across the prepared feature subsets. KNN is useful as a comparator, but its performance is more sensitive to feature-set choice and dimensionality.

From a computational perspective, the pipeline is lightweight enough to run interactively in notebooks. The main cost comes from cross-validation and repeated training across feature sets.

9 Discussion

The results suggest that preprocessing and feature selection have a substantial impact on performance. When feature sets are reduced using tree importance or Lasso, the classifier often performs competitively or better than with less selective subsets. This supports the idea that the dataset contains redundant and weakly informative variables that can be removed without losing useful signal.

The difference between feature-selection strategies is also informative. RFE is objective-driven and tends to preserve model performance, while tree-based methods can surface nonlinear signals that are missed by simple variance filters. Lasso is more aggressive and can produce compact subsets that are easier to inspect, but it may discard features that matter in nonlinear interactions.

The representative confusion matrix shows that the model is good at identifying the majority classes but struggles with one class entirely. This is consistent with class imbalance and suggests that future work should focus on balancing techniques, threshold tuning, and external validation.

10 Limitations

The current study has several limitations:

- The dataset is tabular and derived from prepared feature tables rather than raw images.
- Some classes are underrepresented, which affects minority-class recall.
- Missing values are frequent in the diagnostic fields.
- The project currently uses classical machine learning rather than end-to-end image learning.
- Hardware and execution environment details are not fully recorded in the notebooks.
- The model evaluation is internal to the prepared tables and does not yet include external clinical validation.

11 Ethical Considerations

This project works with anonymized medical imaging data and should be treated as a decision-support experiment rather than a clinical diagnostic tool. Patient privacy depends on the anonymization provided by the source dataset, and any downstream reuse should preserve that anonymization.

Explainability is important in medical AI, so classical models and feature selection are useful because they expose which variables contribute to the decision process. Fairness and bias remain concerns, especially if the underlying cohort is not representative of the broader population.

The system should not be interpreted as a standalone clinical decision engine. Any real-world use would require medical oversight, regulatory review, and strict validation. If future deployment is considered, privacy and compliance requirements such as GDPR or the EU AI Act, where applicable, should be assessed.

12 Future Work

Several extensions are natural next steps:

- Train deep learning models such as CNNs directly on raw CT images.
- Add explainable AI methods such as SHAP or LIME.

- Validate the pipeline on an external dataset.
- Perform more systematic hyperparameter optimization.
- Explore ensemble learning across multiple feature-selection strategies.
- Investigate federated learning for privacy-preserving multi-site analysis.
- Document deployment constraints and clinical integration requirements.

13 Conclusion

This project demonstrates a reproducible classical machine learning workflow for lung cancer classification from CT-derived features. The method combines verification, preprocessing, feature selection, and model evaluation on prepared LIDC-IDRI-derived tables. The experiments show that feature selection materially affects performance and that Random Forest-based approaches are particularly useful for this type of tabular medical data.

The main contribution of the project is a clean baseline pipeline that can be reused and extended. The work shows that classical methods can produce meaningful results on structured CT features, but also highlights the importance of class balance, data quality, and careful validation.

14 References

1. The Cancer Imaging Archive (TCIA). LIDC-IDRI dataset.
2. Kumar et al., radiomics and lung nodule classification literature.
3. scikit-learn documentation.
4. pandas documentation.
5. NumPy documentation.
6. matplotlib documentation.
7. seaborn documentation.

A Repository Structure

```
repository-root/
|-- data/
|   |-- MetaData.csv
|   \-- primary.text
|-- data_0.5.csv
|-- data_1.0.csv
|-- data_1.5.csv
|-- figures/
|-- notebooks/
|   |-- 00_verification/
|   |   \-- verifica.ipynb
|   |-- 01_preprocessing/
|   |   |-- pre_processing.ipynb
|   |   |-- pre_processing2.ipynb
```

```
| | \-- features_pre_processing.ipynb
| |-- 02_feature_selection/
| | \-- feature_selection.ipynb
| \-- 03_modeling/
|     \-- trainingmodels.ipynb
|-- README.md
|-- requirements.txt
\-- TECHNICAL_PAPER.md
```

B Reproducibility

B.1 Installation

```
pip install -r requirements.txt
```

B.2 Dataset Download Instructions

The repository already includes the prepared CSV files used in the notebooks. If you need the raw source data, retrieve it from TCIA and regenerate the prepared tables using the preprocessing notebooks.

B.3 How to Execute the Pipeline

1. Open the notebooks in the `notebooks/` folder.
2. Run `notebooks/00_verification/verifica.ipynb` first.
3. Run the preprocessing notebooks in `notebooks/01_preprocessing/`.
4. Run `notebooks/02_feature_selection/feature_selection.ipynb`.
5. Run `notebooks/03_modeling/trainingmodels.ipynb`.

B.4 Expected Outputs

- Data quality and missing-value reports.
- Selected feature subsets for RFE, tree importance, variance threshold, and Lasso.
- Cross-validation score comparison plots.
- Confusion matrices and class-wise metrics.
- Exported figures in the `figures/` folder.